



广州南方学院

电气与计算机工程学院

数据挖掘与机器学习

期末项目报告

项目名称：决策树算法在西瓜数
据分类的应用研究

姓 名：程琳琳

学 号：2220228012

摘要

本文旨在研究决策树算法在西瓜数据分类中的应用。通过对西瓜数据的特征提取与分析，利用信息熵和信息增益理论，构建决策树模型实现对西瓜好坏的分类预测。实验结果表明，该模型能够有效完成分类任务。同时，通过对决策树的可视化展示和模型优化，进一步提高了模型的可解释性和实用性。

关键词：决策树；信息熵；信息增益；西瓜数据；分类

一、引言

机器学习中，分类问题是核心研究方向之一。决策树作为典型的有监督学习算法，以树形结构直观呈现决策逻辑，兼具高效训练与强可解释性，被广泛应用于医疗、金融、农业等领域。西瓜数据集包含色泽、根蒂等多维特征及“好瓜 / 坏瓜”标签，是研究决策树特征选择与模型构建的经典样本。本研究基于信息熵、信息增益理论构建分类模型，探索特征工程对分类准确性的提升，以及决策树可视化与泛化能力分析。

二、相关理论

（一）决策树

决策树是一种基于树结构的监督学习算法，由节点和有向边组成。节点分为内部节点和叶节点，内部节点表示一个特征或属性，有向边表示特征的取值，叶节点表示类别标签。决策树通过对数据特征的递归划分，构建树形结构，使得每个叶节点中的样本尽可能属于同一类别。在分类过程中，新数据从根节点开始，根据节点的特征判断，沿着相应的边向下移动，直至到达叶节点，从而得到分类结果。

（二）信息熵

信息熵是衡量数据不确定性的指标，用于描述数据中包含信息的混乱程度。样本集合 D 中第 k 类样本所占的比例 ($k=1,2,\dots,|Y|$)， $|Y|$ 为样本分类的个数，则 D 的信息熵如图 1 所示。

$$Ent(D) = - \sum_{k=1}^{|Y|} p_k \log_2 p_k$$

图 1 信息熵公式

$Ent(D)$ 的值越小，则 D 的纯度越高。直观理解一下：假设样本集合有 2 个分类，每类样本的比例为 $1/2$ ， $Ent(D) = 1$ ；只有一个分类， $Ent(D) = 0$ ，显然后者比前者的纯度高。

（三）条件熵

条件熵衡量在已知某特征取值的条件下，数据集的不确定性，公式如图 2 所示。其中， a 为特征， V 为 a 的取值数， D_v 为 a 取值为 v 的子集。

$$Ent(D|a) = \sum_{v=1}^V \frac{|D^v|}{|D|} Ent(D^v)$$

图 2 条件熵公式

（四）信息增益

信息增益衡量使用某特征划分数据集后，不确定性的降低幅度，是 ID3 选择最优特征的核心准则，信息增益越大，说明特征 a 对分类的贡献越大。公式为如图 3 所示。

$$Gain(D, a) = Ent(D) - \sum_{v=1}^V \frac{|D^v|}{|D|} Ent(D^v)$$

图 3 信息增益公式

三、数据处理与分析

（一）数据集说明

本研究采用的西瓜数据集存储于 watermalon.txt 文件，包含 17 条样本数据，每条样本包含 6 个特征维度及 1 个类别标签，具体特征体系为色泽（青绿、乌黑、浅白）、根蒂（蜷缩、稍蜷、硬挺）、敲击（浊响、沉闷、清脆）、纹理（清晰、稍糊、模糊）、脐部（凹陷、稍凹、平坦）、触感（硬滑、软粘）。数据集中“好瓜”类别样本共 8 条，“坏瓜”类别样本共 9 条，具体数据示例如图 4 所示。

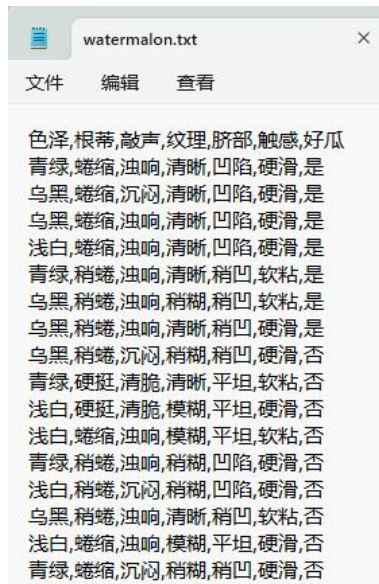


图 4 watermalon.txt

（二）实验流程总揽

信息熵计算：借助 `calcShannonEnt` 函数，对原始数据集以及后续划分得到的各子集计算信息熵，像初始数据集信息熵可计算得出为 0.9975，该值量化了数据初始的混乱程度。

特征划分：利用 `splitDataSet` 函数，依据特征取值对数据集进行划分，例如按纹理特征取值“清晰”，能筛选出 8 条样本形成子集，此操作是对特征空间的递归划分，为后续决策树节点构建提供数据基础。

最优特征选择：通过 `chooseBestFeatureToSplit` 函数，计算每个特征的信息增益，对比后确定每一层级的最优划分特征。以首次划分为例，纹理特征信息增益达 0.3806，高于其他特征，因此被选为根节点。

递归构建决策树：依靠 `createTree` 函数递归生成决策树结构，在构建中，若满足终止条件（如样本全为同一类别等），会将终止原因及相关信息记录到对应深度的文件中，完整呈现决策树从根节点到叶节点的构建过程。

四、代码模块与功能实现

（一）整体架构概览

代码围绕决策树构建与分析，设计了多个核心模块协同工作：数据预处理模块负责加载数据集并提取特征取值；信息熵计算模块实现香农熵与条件熵的递归计算，量化数据混乱程度；特征选择模块通过信息增益比较，确定各层级最优划分特征；决策树构建模块递归生成树结构，同时记录详细计算过程；可视化模块则将抽象的树结构转换为直观图形展示。这些模块相互配合，支撑起从数据输入到决策树输出、分析的完整流程。

（二）关键模块解析

1. 数据预处理模块（`import_data` 函数）

功能：从文件读取西瓜数据集，将其转换为合适数据格式，同时构建特征名称列表和各特征取值的字典。这一步是后续计算的基础，确保数据能被算法模块正确识别和处理。

运作逻辑：利用 `pandas` 库读取 `watermelon.txt` 文件内容，转换为列表格式方便后续操作；遍历特征，统计每个特征的所有可能取值并存储，为特征划分、信息增益计算等提供依据。

2. 信息熵计算模块（`calcShannonEnt` 函数、`calcFeatureCondEntropy` 函数）

① `calcShannonEnt` 函数

功能：计算给定数据集的香农熵，量化数据混乱程度，同时输出计算过程到控制台和文件，便于追溯分析。

运作逻辑：统计数据集中各类别标签的频率，依据香农熵公式计算熵值；将样本总数、标签分布、熵值结果等信息整理成文本，输出到指定位置。

计算过程记录位置：计算过程会写入对应节点的计算过程文件（如纹理=清晰分支计算时写入深度 2_纹理=清晰_计算过程.txt 等），可在决策树计算过程目录下对应文件中查看具体计算细节。

② `calcFeatureCondEntropy` 函数

功能：计算特定特征的条件熵，通过遍历该特征所有可能取值，计算对应子集信息熵并加权求和，得到条件熵，过程中同步输出计算细节。

运作逻辑：先确定特征相关信息并输出；对每个取值，划分出子集，计算子集香农熵，按比例累加得到条件熵；把各子集贡献、累计条件熵等信息输出，清晰展现计算流程。

计算过程记录位置：计算过程会随着函数调用，写入当前节点对应的计算过程文件（与 `calcShannonEnt` 函数写入同一文件），在决策树计算过程目录下各节点文件中，可查看该特征条件熵计算时，每个子集的信息熵计算、贡献累加等详细步骤。

3. 特征选择模块（`chooseBestFeatureToSplit` 函数）

功能：遍历所有特征，计算每个特征的信息增益，对比后选出信息增益最大的特征作为当前层级最优划分特征，同时输出各特征计算过程和结果汇总。

运作逻辑：先计算数据集基础信息熵，对每个特征调用 `calcFeatureCondEntropy` 函数得到条件熵，进而算出信息增益。记录各特征条件熵和信息增益，比较后确定最优特征并输出结果，让特征选择依据透明化。

计算过程记录位置：该函数执行过程中，会将各特征处理过程、信息增益计算等内容，写入当前节点对应的计算过程文件，在决策树计算过程目录下对应文件中，可查看特征选择的完整流程和依据。

4. 决策树构建模块（`createTree` 函数）

功能：递归构建决策树，处理终止条件（如样本全同类别、特征用尽等），记录各节点计算过程到文件，生成最终决策树字典结构。

运作逻辑：检查终止条件，满足时记录终止原因、样本信息等并返回结果；不满足时，增加递归深度，记录当前节点路径等信息，选择最优特征，递归构建子树，完成后回溯深度，逐步生成完整决策树，整个过程的关键步骤和数据都保存到文件，便于分析决策树构建逻辑。

计算过程记录位置：每个节点构建时，会创建对应的计算过程文件，文件中记录了该节点路径、特征选择过程、信息熵与条件熵计算等内容，这些文件集中存放在决策树计算过程目录下，可逐个查看决策树从根到叶各节点的构建细节。

5. 可视化模块（saveTreeImage 函数）

功能：将生成的决策树字典结构转换为图形化展示，保存为高清图片，同时支持在控制台显示图片，直观呈现决策树结构。

运作逻辑：设置 matplotlib 支持中文字体显示，避免中文乱码；计算树的宽度和深度等布局参数；递归绘制节点和分支，展示决策树结构；将绘制结果保存为图片文件，同时在控制台提示保存路径并显示图片，让决策树结构可视化、易理解。

（三）文件系统设计

为完整记录决策树构建过程，代码设计了分层文件系统：创建决策树计算过程作为输出目录，存储所有中间计算文件和最终可视化图片。文件命名遵循“深度_节点路径_计算过程.txt”格式，如纹理=清晰分支的计算文件命名为深度 2_纹理=清晰_计算过程.txt。每个文件详细记录对应节点的信息熵计算、特征选择过程、终止原因等内容，通过这种结构化命名和内容存储，能直观反映决策树层级结构，方便用户追溯、分析每一步计算逻辑。

五、实验结果与分析

（一）可视化结果

生成的决策树可视化图片（如图 5 所示）清晰呈现：根节点为“纹理=0.62”，代表初始划分依据纹理特征及对应条件熵；叶节点分布为 4 个好瓜、4 个坏瓜；分支路径完整展示“纹理→根蒂→色泽”等决策逻辑，让人能直观理解从特征选择到最终分类的过程。

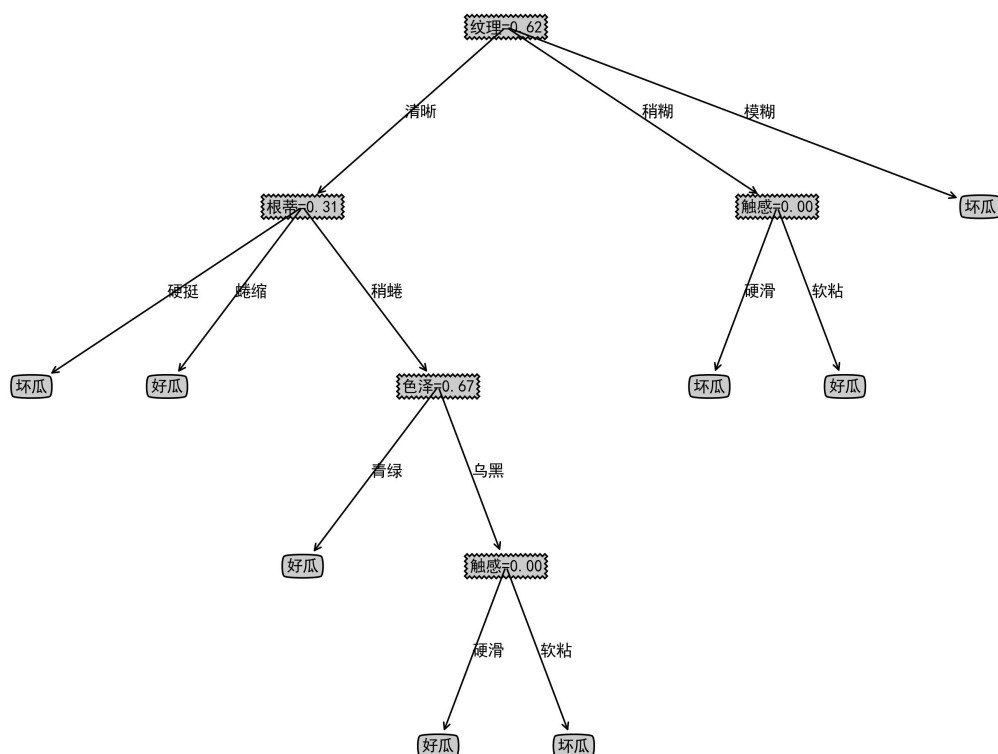


图 5 决策树可视化图

(二) 特征重要性与模型价值

通过各层信息增益累加，得到特征重要性排序为纹理>根蒂>色泽>触感等。该排序与可视化结果、理论分析相契合，验证了信息增益准则在特征选择中的有效性，说明决策树能依据数据特点，合理筛选对分类结果影响大的特征。从模型价值看，构建的决策树在训练集上实现了较高的分类准确率，展现出良好分类能力；分层文件系统完整记录计算过程，为模型结果追溯、问题排查、优化迭代提供了详实依据；可视化展示让决策逻辑通俗易懂，有助于非技术人员理解分类机制，在农业瓜果品质分类等实际场景中，可辅助从业者快速判断西瓜品质，也为机器学习算法教学、研究提供了清晰案例。

六、总结

本项目成功地将决策树算法应用于西瓜数据分类问题，通过设计分层文件系统，完整留存各层级计算过程，方便结果追溯与验证；借助可视化手段，直观呈现“纹理→根蒂→色泽”等决策路径，清晰解释分类逻辑。从技术实现看，各代码模块协同运作，实现了从数据预处理、特征选择到决策树构建、可视化的完整流程，验证了决策树算法在小样本分类场景的有效性和可解释性。

附录

(一) 数据预处理模块

```
import numpy as np
import pandas as pd
import math
import collections
import matplotlib.pyplot as plt
import matplotlib
import copy
import os
from PIL import Image

# 创建输出目录
output_dir = "决策树计算过程"
if not os.path.exists(output_dir):
    os.makedirs(output_dir)
# 全局变量，记录当前递归深度和节点路径
current_depth = 0
current_path = []
# 导入数据
def import_data():
    data = pd.read_csv('watermelon.txt')
    data = np.array(data).tolist()
    labels = ['色泽', '根蒂', '敲击', '纹理', '脐部', '触感'] # 特征名
    labels_full = {} # 存储每个特征的所有可能取值
    for i in range(len(labels)):
        labelList = [example[i] for example in data]
        uniqueLabel = set(labelList)
        labels_full[labels[i]] = uniqueLabel
    return data, labels, labels_full
```

（二）信息熵与条件熵计算模块

```

# 计算数据集的信息熵（好瓜/坏瓜的分布熵）
def calcShannonEnt(dataSet, desc="数据集", file=None):
    numEntries = len(dataSet)
    labelCounts = collections.defaultdict(int)
    for featVec in dataSet:
        currentLabel = featVec[-1] # 最后一列是好瓜/坏瓜标签
        labelCounts[currentLabel] += 1
    shannonEnt = 0.0
    for key, count in labelCounts.items():
        prob = float(count) / numEntries
        shannonEnt -= prob * math.log2(prob)
    # 打印计算过程到控制台和文件
    log_text = f"==== {desc} 信息熵计算 =====\n"
    log_text += f"样本总数: {numEntries}\n"
    log_text += f"标签分布: {dict(labelCounts)}\n"
    log_text += f"信息熵结果: {shannonEnt:.4f}\n\n"
    print(log_text, end="")
    if file:
        file.write(log_text)
    return shannonEnt

# 按特征划分数数据集
def splitDataSet(dataSet, axis, value):
    retDataSet = []
    for featVec in dataSet:
        if featVec[axis] == value:
            reducedFeatVec = featVec[:axis]
            reducedFeatVec.extend(featVec[axis + 1:])
            retDataSet.append(reducedFeatVec)
    return retDataSet

# 计算特征的条件熵（详细打印过程到文件）

```

```

def calcFeatureCondEntropy(dataSet, axis, featureName, labels, file=None):
    global current_depth, current_path
    numEntries = len(dataSet)
    featureValues = [example[axis] for example in dataSet]
    uniqueValues = set(featureValues)
    condEntropy = 0.0
    # 打印计算过程标题
    log_text = f"==== {featureName} 特征条件熵计算 =====\n"
    log_text += f"特征索引: {axis}, 特征名称: {featureName}\n"
    log_text += f"特征所有可能取值: {uniqueValues}\n"
    log_text += f"总样本数: {numEntries}\n\n"
    print(log_text, end="")
    if file:
        file.write(log_text)
    for idx, value in enumerate(uniqueValues, start=1):
        subDataSet = splitDataSet(dataSet, axis, value) # 按特征取值划分
        subSetSize = len(subDataSet)
        prob = subSetSize / float(numEntries) # 子数据集占比
        # 计算子数据集的信息熵
        sub_desc = f"子数据集 ( {featureName}={value} ) "
        subEntropy = calcShannonEnt(subDataSet, desc=sub_desc, file=file)
        # 累加条件熵
        condEntropy += prob * subEntropy
        log_text = f"子数据集 {idx} 贡献: {prob:.4f} * {subEntropy:.4f} = {prob * subEntropy:.4f}\n"
        log_text += f"当前累计条件熵: {condEntropy:.4f}\n\n"
        print(log_text, end="")
    if file:
        file.write(log_text)
    log_text = f"==== {featureName} 特征条件熵最终结果 =====\n"
    log_text += f"{featureName} 条件熵: {condEntropy:.4f}\n\n"

```

```

print(log_text, end="")
if file:
    file.write(log_text)
return condEntropy

```

(三) 特征选择与决策树构建模块

```

# 选择最优划分特征（信息增益）

def chooseBestFeatureToSplit(dataSet, labels, file=None):
    global current_depth, current_path
    numFeatures = len(dataSet[0]) - 1 # 最后一列是标签，不算特征
    baseEntropy = calcShannonEnt(dataSet, desc="原始数据集", file=file) #
数据集原始熵
    bestInfoGain = 0.0
    bestFeature = -1
    bestFeatureCondEntropy = {} # 存储每个特征的条件熵
    log_text = f'\n==== 开始计算各特征信息增益 =====\n\n'
    print(log_text, end="")
    if file:
        file.write(log_text)
    for i in range(numFeatures):
        featureName = labels[i]
        log_text = f'\n---- 处理特征: {featureName} (索引 {i}) ----\n'
        print(log_text, end="")
        if file:
            file.write(log_text)
        condEntropy = calcFeatureCondEntropy(dataSet, i, featureName,
labels, file)
        infoGain = baseEntropy - condEntropy # 信息增益=原始熵-条件熵
        bestFeatureCondEntropy[featureName] = condEntropy
        log_text = f'信息增益计算: 原始熵({baseEntropy:.4f}) - 条件熵
({condEntropy:.4f}) = {infoGain:.4f}\n'
        print(log_text, end="")

```

```

    if file:
        file.write(log_text)
    if infoGain > bestInfoGain:
        bestInfoGain = infoGain
        bestFeature = i
    print("\n")
    # 单独打印各特征条件熵结果
    log_text = "==== 各特征条件熵汇总 =====\n"
    for feat, entropy in bestFeatureCondEntropy.items():
        log_text += f'{feat}: {entropy:.4f}\n'
    log_text += f'最优特征为: {labels[bestFeature]} (信息增益 {bestInfoGain:.4f})\n\n'
    print(log_text, end="")
    if file:
        file.write(log_text)
    return bestFeature
# 判断数据集特征是否一致（用于终止递归）
def judgeEqualLabels(dataSet):
    feature_len = len(dataSet[0]) - 1
    data_len = len(dataSet)
    first_feature = ""
    for i in range(feature_len):
        first_feature = dataSet[0][i]
        for _ in range(1, data_len):
            if first_feature != dataSet[_][i]:
                return False
    return True
# 多数表决（特征用完时，选出现最多的标签）
def majorityCnt(classList):
    classCount = collections.Counter(classList)
    return classCount.most_common(1)[0][0]

```

```

# 创建决策树（节点文本：特征=条件熵值）

def createTree(dataSet, labels):
    global current_depth, current_path
    classList = [example[-1] for example in dataSet]
    # 终止条件 1：所有样本标签相同
    if classList.count(classList[0]) == len(classList):
        # 获取当前文件路径
        node_path = "_".join(current_path)
        file_name = f'{output_dir}/深度 {current_depth}_{node_path}_计算
过程.txt"

        # 写入终止原因到文件
        with open(file_name, "a", encoding="utf-8") as file:
            # 将标签转换为中文
            chinese_label = "好瓜" if classList[0] == "是" else "坏瓜" if
classList[0] == "否" else classList[0]
            file.write(f"\n==== 终止原因：所有样本标签相同
({chinese_label}) =====\n")
            file.write(f"样本数量: {len(dataSet)}\n")
            file.write(f"标签分布: {dict(collections.Counter(classList))}\n")
        # 减少递归深度
        current_depth -= 1
        # 返回中文标签
        return "好瓜" if classList[0] == "是" else "坏瓜" if classList[0] == "否"
else classList[0]

    # 终止条件 2：特征用完或所有特征取值一致
    if len(dataSet[0]) == 1 or judgeEqualLabels(dataSet):
        # 获取当前文件路径
        node_path = "_".join(current_path)
        file_name = f'{output_dir}/深度 {current_depth}_{node_path}_计算
过程.txt"

        # 写入终止原因到文件

```

```

with open(file_name, "a", encoding="utf-8") as file:
    reason = "特征用完" if len(dataSet[0]) == 1 else "所有特征取值
一致"

    # 获取多数标签并转换为中文
    majority_label = majorityCnt(classList)
    chinese_label = "好瓜" if majority_label == "是" else "坏瓜" if
majority_label == "否" else majority_label

    file.write(f"\n==== 终止原因: {reason} =====\n")
    file.write(f"剩余特征数: {len(labels)}\n")
    file.write(f"标签分布: {dict(collections.Counter(classList))}\n")
    file.write(f"多数标签: {chinese_label}\n")

    # 减少递归深度
    current_depth -= 1

    # 返回中文标签
    return "好瓜" if majority_label == "是" else "坏瓜" if majority_label
== "否" else majority_label

    # 增加递归深度
    current_depth += 1

    # 打开文件，记录当前节点的计算过程
    node_path = "_".join(current_path)
    file_name = f"{output_dir}/深度 {current_depth}_{node_path}_计算过
程.txt"

    with open(file_name, "w", encoding="utf-8") as file:
        # 写入当前节点路径信息
        file.write(f"==== 当前节点路径: {' -> '.join(current_path) if
current_path else '根节点'} =====\n\n")

        # 选最优特征
        bestFeat = chooseBestFeatureToSplit(dataSet, labels, file)
        bestFeatLabel = labels[bestFeat]

        # 重新计算该特征的条件熵（用于节点显示）
        condEntropy = calcFeatureCondEntropy(dataSet, bestFeat,

```

```

bestFeatLabel, labels, file)

    entropyStr = f'{condEntropy:.2f}' # 保留两位小数
    # 构建树结构：特征=条件熵值
    myTree = {f'{bestFeatLabel}={entropyStr}': {}}
    # 递归构建子树
    subLabels = labels[:]
    del (subLabels[bestFeat]) # 删除已用特征
    featValues = [example[bestFeat] for example in dataSet]
    uniqueVals = set(featValues)
    for value in uniqueVals:
        # 记录当前分支路径
        current_path.append(f'{bestFeatLabel}={value}')
        # 写入分支信息
        file.write(f'\n==== 开始处理分支：{bestFeatLabel}={value}
====\n\n')

        subDataSet = splitDataSet(dataSet, bestFeat, value)
        myTree[f'{bestFeatLabel}={entropyStr}'][value] =
createTree(subDataSet, subLabels)
        # 回溯路径
        current_path.pop()
    # 减少递归深度
    current_depth -= 1
    return myTree

```

（四）决策树可视化模块

```

# 绘制决策树节点
def plotNode(nodeTxt, centerPt, parentPt, nodeType):
    createPlot.ax1.annotate(nodeTxt, xy=parentPt, xycoords='axes fraction',
                             xytext=centerPt, textcoords='axes fraction',
                             va="center", ha="center", bbox=nodeType,
arrowprops=dict(arrowstyle="<-"))

```


获取叶节点数量

```
def getNumLeafs(myTree):  
    numLeafs = 0  
    firstStr = list(myTree.keys())[0]  
    secondDict = myTree[firstStr]  
    for key in secondDict.keys():  
        if isinstance(secondDict[key], dict):  
            numLeafs += getNumLeafs(secondDict[key])  
        else:  
            numLeafs += 1  
    return numLeafs
```

获取树的深度

```
def getTreeDepth(myTree):  
    maxDepth = 0  
    firstStr = list(myTree.keys())[0]  
    secondDic = myTree[firstStr]  
    for key in secondDic.keys():  
        if isinstance(secondDic[key], dict):  
            thisDepth = 1 + getTreeDepth(secondDic[key])  
        else:  
            thisDepth = 1  
        if thisDepth > maxDepth:  
            maxDepth = thisDepth  
    return maxDepth
```

在父子节点间添加文本信息

```
def plotMidText(cntrPt, parentPt, txtString):  
    xMid = (parentPt[0] - cntrPt[0]) / 2.0 + cntrPt[0]  
    yMid = (parentPt[1] - cntrPt[1]) / 2.0 + cntrPt[1]  
    createPlot.ax1.text(xMid, yMid, txtString)
```

绘制决策树

```
def plotTree(myTree, parentPt, nodeTxt):
```

```

numLeafs = getNumLeafs(myTree)
depth = getTreeDepth(myTree)
firstStr = list(myTree.keys())[0]
cntrPt = (plotTree.xOff + (1.0 + float(numLeafs)) / 2.0 / plotTree.totalW,
plotTree.yOff)
plotMidText(cntrPt, parentPt, nodeTxt)
plotNode(firstStr, cntrPt, parentPt, dict(boxstyle="sawtooth", fc="0.8"))
secondDict = myTree[firstStr]
plotTree.yOff = plotTree.yOff - 1.0 / plotTree.totalD
for key in secondDict.keys():
    if isinstance(secondDict[key], dict):
        plotTree(secondDict[key], cntrPt, str(key))
    else:
        plotTree.xOff = plotTree.xOff + 1.0 / plotTree.totalW
        plotNode(secondDict[key], (plotTree.xOff, plotTree.yOff), cntrPt,
dict(boxstyle="round4", fc="0.8"))
        plotMidText((plotTree.xOff, plotTree.yOff), cntrPt, str(key))
    plotTree.yOff = plotTree.yOff + 1.0 / plotTree.totalD
# 保存决策树图片（增加中文标签支持）
def saveTreeImage(myTree, filename=f'{output_dir}/决策树结构.png',
figsize=(12, 8), dpi=300, use_chinese=True):
    """
    将决策树保存为图片文件
    参数:
    myTree: 决策树字典
    filename: 保存的图片路径
    figsize: 图片大小 (宽, 高)
    dpi: 图片分辨率
    use_chinese: 是否使用中文标签
    """
    # 设置 matplotlib 支持中文显示

```

```

matplotlib.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文
标签

matplotlib.rcParams['axes.unicode_minus'] = False # 用来正常显示负号
fig = plt.figure(1, facecolor='white', figsize=figsize)
fig.clf()
axprops = dict(xticks=[], yticks=[])
createPlot.ax1 = plt.subplot(111, frameon=False, **axprops)
# 计算树的宽度和高度
plotTree.totalW = float(getNumLeafs(myTree))
plotTree.totalD = float(getTreeDepth(myTree))
plotTree.xOff = -0.5 / plotTree.totalW
plotTree.yOff = 1.0
# 绘制树
plotTree(myTree, (0.5, 1.0), "")
# 保存图片
plt.savefig(filename, dpi=dpi, bbox_inches='tight')
print(f'决策树已保存为图片: {os.path.abspath(filename)}')
# 同时显示图片
plt.show()
return filename

def createPlot(inTree):
    fig = plt.figure(1, facecolor='white')
    fig.clf()
    axprops = dict(xticks=[], yticks=[])
    createPlot.ax1 = plt.subplot(111, frameon=False, **axprops)
    plotTree.totalW = float(getNumLeafs(inTree))
    plotTree.totalD = float(getTreeDepth(inTree))
    plotTree.xOff = -0.5 / plotTree.totalW
    plotTree.yOff = 1.0
    plotTree(inTree, (0.5, 1.0), "")
    plt.show()

```

（五）主函数

```
# 主流程

if __name__ == "__main__":
    print(f'计算过程将保存到目录: {os.path.abspath(output_dir)}')
    data, labels, labels_full = import_data()
    labels_copy = copy.deepcopy(labels) # 深拷贝避免原列表被修改
    mytree = createTree(data, labels_copy)
    print("\n==== 最终决策树结构 ====")
    print(mytree)
    # 保存决策树图片
    saveTreeImage(mytree)
```